MEPCO SCHLENK ENGINEERING COLLEGE, SIVAKASI
(AUTONOMOUS)
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

NAME : ……..

Class & Dept :……

Roll No. : ……..

Lab : ……

LABORATORY RECORD

***Register No*: ……**

CERTIFICATE

This is to Certify that it is the Bonafide work done by…… in the 23AD653 MINI PROJECT - IV : NATURAL LANGUAGE PROCESSING at Mepco Schlenk Engineering College, Sivakasi during the year 2025– 2026

…… ……

Faculty in Charge ***Head of the Department***

Submitted for the Practical Examination held at Mepco Schlenk Engineering College, Sivakasi on …….

……. ……

Internal Examiner ***External Examiner***

CONTENTS

Exp No.	Name of the Exercise	Page No.	Date		Marks	Faculty Sign
			Exercise	Submission		
1.	Module1: Basic Text Processing					
2.	Module 2: Morphology And Language Models					
3.	Module 3: Chunking And Corpora Analysis					
4.	Module 4: Named Entity & Parsing Techniques					
5.	Module 5: Advanced NLP Techniques					
6.	Module 6: NLP Case Studies & Transformers					

Date:	MODULE 1: BASIC TEXT PROCESSING

Objective:

- To understand how raw text is processed
- To implement tokenization (word/sentence level)
- To validate text scripts (identify language/character patterns)

Problem Statement:

Given a raw text document, perform:

1. Basic text operations (reading, cleaning)
2. Tokenization
3. Script validation to identify valid/invalid text patterns

Tools Required:

- Python 3.x
- Libraries:
    - nltk
    - re (Regular Expressions)

Description:

1. Text Processing

Text Processing is the first and most important step in Natural Language Processing (NLP). Real-world text data is usually unstructured, noisy, and inconsistent. Before applying any advanced NLP techniques, the text must be cleaned and transformed into a structured format.

Text processing helps in:

- Improving data quality
- Reducing noise
- Making text suitable for machine learning models

2. Basic Text Processing Operations

Basic text processing includes simple operations that prepare the text for further analysis.

2.1 Case Conversion

- Converts text into lowercase or uppercase
- Helps maintain uniformity

Example:
"AI is Powerful" → "ai is powerful"

Why important?
Because “AI” and “ai” should be treated as the same word.

2.2 Character Count

- Counts total number of characters including spaces
- Helps in analyzing text size

2.3 Word Count

- Counts number of words in a sentence
- Used in text analytics and statistics

2.4 Removing Punctuation

- Eliminates special symbols such as:
    - ! , . ? ; :
- These symbols usually do not contribute to meaning in many NLP tasks

Example:
"Hello!" → "Hello"

2.5 String Manipulation

Includes:

- Splitting text (split())
- Joining text
- Replacing words

3. Text Pre-processing

Text preprocessing is a set of techniques used to clean and transform raw text into a meaningful format.

It includes:

- Tokenization
- Script validation

- Stopword removal (next modules)
- Stemming / Lemmatization (later modules)

4. Tokenization

Definition:

Tokenization is the process of breaking text into smaller units called tokens.

Types of Tokenization

4.1 Sentence Tokenization

Splits text into sentences.

Example:
"AI is growing. It is powerful."
→ ["AI is growing.", "It is powerful."]

4.2 Word Tokenization

Splits sentences into words.

Example:
"AI is powerful"
→ ["AI", "is", "powerful"]

Importance of Tokenization

- Forms the base for all NLP tasks
- Required for:
    - POS tagging
    - Parsing
    - Machine learning models

5. Script Validation

Definition:

Script validation is the process of checking whether the text contains valid characters or not.

Need for Script Validation

Text data may contain:

- Special characters (!, @, #, etc.)
- Numbers
- Mixed scripts

These may affect NLP performance, so they are filtered.

Methods for Script Validation

5.1 Using isalpha()

- Checks if word contains only alphabets

Example:
"AI" → Valid
"AI123" → Invalid

5.2 Using Regular Expressions (Regex)

Regex is used for pattern matching.

Example pattern:
^[A-Za-z]+$

- Allows only alphabets
- Removes unwanted tokens

6. Importance of Pre-processing in NLP

Preprocessing improves:

- Accuracy of models
- Efficiency of algorithms
- Quality of insights

Without preprocessing:

- Data becomes noisy
- Models perform poorly

7. Applications

- Chatbots
- Spell checking
- Search engines
- Sentiment analysis
- Machine translation

8. Advantages

- Reduces noise in data
- Improves consistency
- Enhances model performance

9. Limitations

- Over-cleaning may remove useful information
- Language-dependent challenges
- Requires careful design

Conclusion:

The fundamental text preprocessing techniques were implemented to convert raw text into a structured format. Tokenization enabled the breakdown of text into meaningful units, while script validation ensured data consistency and correctness, supporting effective further NLP processing.

Date:	MODULE 2: MORPHOLOGY AND LANGUAGE MODELS

Objective:

• To understand morphological analysis in NLP
• To explore language models using the n-gram approach
• To implement Part-of-Speech (POS) tagging

Problem Statement:

Given a text document, perform:

1. Morphological analysis to identify root words and variations
2. Build and analyze n-gram language models
3. Apply POS tagging to identify grammatical categories of words

Tools Required:

• Python 3.x
• Libraries:
o nltk
o re (Regular Expressions)

Description:

1. Morphological Analysis

Morphological analysis is the study of the internal structure of words. It focuses on identifying the root word (stem) and its variations such as prefixes and suffixes.

Morphology helps in:
• Understanding word formation
• Reducing words to their base form
• Improving text normalization

2. Types of Morphology

2.1 Inflectional Morphology

• Modifies a word to express grammatical features
• Does not change the core meaning

Example:
play → plays, played, playing

2.2 Derivational Morphology

• Forms a new word with a different meaning
• Often changes the word class

Example:
happy → happiness

3. Stemming and Lemmatization

3.1 Stemming

• Reduces words to their root form by removing suffixes
• May not produce meaningful words

Example:
running → run

3.2 Lemmatization

• Converts words into their base dictionary form
• Produces meaningful words

Example:
better → good

4. Language Models

A language model predicts the probability of a sequence of words. It plays an important role in many NLP applications.

5. n-Gram Model

Definition:
An n-gram is a sequence of ‘n’ words used to predict the next word in a sentence.

Types of n-Gram

5.1 Unigram (n = 1)

• Single word
Example:
["AI", "is", "powerful"]

5.2 Bigram (n = 2)

• Sequence of two words
Example:
["AI is", "is powerful"]

5.3 Trigram (n = 3)

• Sequence of three words
Example:
["AI is powerful"]

Importance of n-Gram Models

• Helps in text prediction
• Used in speech recognition
• Improves machine translation

6. Part-of-Speech (POS) Tagging

Definition:
POS tagging is the process of assigning grammatical labels to each word in a sentence.

Common POS Tags

• Noun (NN)
• Verb (VB)
• Adjective (JJ)
• Adverb (RB)

Example

Sentence:
"AI is powerful"

Output:
AI → Noun

is → Verb
powerful → Adjective

7. Importance of POS Tagging

• Helps in understanding sentence structure
• Used in parsing and syntax analysis
• Improves NLP model performance

8. Applications

• Chatbots
• Machine translation
• Speech recognition
• Text prediction
• Grammar checking

9. Advantages

• Improves language understanding
• Helps in better text analysis
• Supports advanced NLP tasks

10. Limitations

• Ambiguity in word meanings
• Context-dependent tagging errors
• Requires large datasets for accuracy

Conclusion:

The essential concepts of morphological analysis, language modeling using the n-gram approach, and part-of-speech tagging were implemented to analyze and understand the structure of language. Morphological analysis helped in identifying root words and their variations, while the n-gram model enabled prediction of word sequences. POS tagging identified grammatical categories, enhancing sentence structure understanding for further NLP tasks.

Date:	MODULE 3: CHUNKING AND CORPORA ANALYSIS

Objective:

• To explore chunking techniques for extracting noun phrases
• To understand and explore various language corpora
• To analyze words using WordNet and perform different operations

Problem Statement:

Given a text document, perform:

1. Chunking to extract noun phrases
2. Exploration of different language corpora
3. Word analysis using WordNet (synonyms, antonyms, meanings)

Tools Required:

• Python 3.x
• Libraries:
o nltk
o wordnet (from nltk.corpus)

Description:

1. Chunking

Chunking is a process used in Natural Language Processing to group words into meaningful phrases based on their grammatical structure. It is also known as shallow parsing.

Chunking helps in:
• Extracting important phrases from text
• Understanding sentence structure
• Improving information extraction

2. Noun Phrase (NP) Chunking

Noun Phrase Chunking identifies groups of words that act as a noun in a sentence.

Example

Sentence:
"The quick brown fox jumps over the lazy dog"

Noun Phrases:
• The quick brown fox
• the lazy dog

3. Chunking Process

Steps involved:

1. Perform tokenization
2. Apply POS tagging
3. Define chunk grammar using rules
4. Extract chunks

4. Chunk Grammar

Chunk grammar defines patterns for extracting phrases.

Example:
NP → Determiner + Adjective + Noun

5. Language Corpora

A corpus is a large and structured collection of text used for linguistic analysis and NLP tasks.

Types of Corpora

5.1 General Corpus

• Contains text from various domains
• Example: newspapers, books

5.2 Specialized Corpus

• Domain-specific text
• Example: medical or legal documents

5.3 Annotated Corpus

• Contains labeled data (POS tags, syntax)
• Used for training NLP models

Importance of Corpora

• Provides real-world language data
• Helps in training NLP models
• Used for testing and evaluation

6. WordNet

WordNet is a lexical database of the English language that groups words into sets of synonyms called synsets.

Features of WordNet

• Synonyms (similar words)
• Antonyms (opposite words)
• Definitions (meanings)
• Example sentences

7. WordNet Operations

7.1 Synonyms

Words with similar meaning
Example:
happy → joyful

7.2 Antonyms

Words with opposite meaning
Example:
happy → sad

7.3 Hypernyms

• Generalized terms
Example:
dog → animal

7.4 Hyponyms

• Specific terms
Example:
animal → dog

8. Importance of WordNet

• Improves semantic understanding
• Helps in word sense disambiguation
• Enhances search and retrieval systems

9. Applications

• Information extraction
• Chatbots
• Search engines
• Text classification
• Semantic analysis

10. Advantages

• Provides structured lexical knowledge
• Improves NLP accuracy
• Supports semantic analysis

11. Limitations

• Limited to specific languages (mainly English)
• May not capture context perfectly
• Requires integration with other techniques

Conclusion:

Chunking techniques were used to extract noun phrases, enabling better understanding of sentence structure. Language corpora provided a rich source of real-world text data for analysis, while WordNet facilitated semantic analysis through synonyms, antonyms, and hierarchical relationships. These approaches collectively enhance the ability to process and interpret natural language effectively.

Date:	MODULE 4: NAMED ENTITY & PARSING TECHNIQUES

Objective:

• To implement Named Entity Recognition (NER)
• To explore various parsing techniques
• To understand and apply Word Sense Disambiguation (WSD) techniques

Problem Statement:

Given a text document, perform:

1. Named Entity Recognition to identify entities such as names, locations, and organizations
2. Apply parsing techniques to analyze sentence structure
3. Perform word sense disambiguation to determine correct meanings of words

Tools Required:

• Python 3.x
• Libraries:
o nltk
o spacy (optional)

Description:

1. Named Entity Recognition (NER)

Named Entity Recognition is a process in Natural Language Processing used to identify and classify important elements in text into predefined categories.

NER identifies:
• Person names
• Locations
• Organizations
• Dates and time

Example

Sentence:
"Ravi works at TCS in Chennai"

Output:
Ravi → Person

TCS → Organization
Chennai → Location

Importance of NER

• Helps in information extraction
• Used in search engines
• Improves chatbot responses
• Useful in recommendation systems

2. Parsing

Parsing is the process of analyzing the grammatical structure of a sentence based on a set of rules.

Parsing helps in:
• Understanding sentence structure
• Identifying relationships between words
• Supporting syntax analysis

3. Types of Parsing

3.1 Top-Down Parsing

• Starts from the root node and moves downward
• Attempts to match the input sentence

3.2 Bottom-Up Parsing

• Starts from the input words and builds up to the root
• Combines words into phrases

3.3 Dependency Parsing

• Focuses on relationships between words
• Represents sentence as dependencies

Example

Sentence:
"She eats an apple"

Parsing identifies subject, verb, and object relationships.

4. Word Sense Disambiguation (WSD)

Word Sense Disambiguation is the process of determining the correct meaning of a word based on context.

Example

Word: "bank"
• River bank
• Financial bank

WSD identifies the correct meaning based on context.

5. WSD Techniques

5.1 Knowledge-Based Methods

• Uses dictionaries or lexical databases
• Example: WordNet

5.2 Supervised Methods

• Uses labeled training data
• Machine learning approaches

5.3 Unsupervised Methods

• Uses unlabeled data
• Clustering techniques

6. Importance of WSD

• Improves text understanding
• Reduces ambiguity
• Enhances NLP applications

7. Applications

• Chatbots
• Machine translation
• Search engines
• Question answering systems
• Information retrieval

8. Advantages

• Improves semantic understanding
• Enhances accuracy of NLP models
• Helps in better decision-making

9. Limitations

• Complex to implement
• Requires large datasets
• Context ambiguity remains a challenge

Conclusion

Named Entity Recognition enabled the identification of important entities such as persons, locations, and organizations from text. Parsing techniques helped in analyzing sentence structure and understanding relationships between words. Word Sense Disambiguation resolved ambiguity by identifying the correct meaning of words based on context, thereby improving the overall effectiveness of natural language processing tasks.

Date:	MODULE 5: ADVANCED NLP TECHNIQUES

Objective:

• To implement coreference resolution algorithms
• To understand and apply word embedding techniques

Problem Statement:

Given a text document, perform:

1. Coreference resolution to identify related entities across sentences
2. Apply word embedding techniques to represent words numerically

Tools Required:

• Python 3.x
• Libraries:
o nltk
o spacy
o gensim

Description:

1. Coreference Resolution

Coreference resolution is the process of identifying when different words refer to the same entity in a text.

Example

Sentence:
"Ravi went to the store. He bought milk."

Here,
He → Ravi

Importance of Coreference Resolution

• Improves text understanding
• Helps in information extraction
• Useful in chatbots and summarization

2. Types of Coreference

• Pronoun resolution (he, she, it)
• Noun phrase resolution
• Anaphora resolution

3. Word Embeddings

Word embeddings are numerical vector representations of words that capture their meanings, relationships, and context. Traditional methods treat words as independent units, whereas embeddings capture semantic similarity.

Example:
king ≈ queen
man ≈ woman

4. Types of Word Embedding Techniques

4.1 Word2Vec

• Predicts words based on context
• Two models:
o CBOW (Continuous Bag of Words)
o Skip-gram

4.2 GloVe (Global Vectors)

• Uses global word co-occurrence statistics
• Captures overall word relationships

4.3 FastText

• Considers subword information
• Works well with rare words

5. Applications

• Text similarity
• Recommendation systems
• Sentiment analysis
• Machine translation

6. Advantages

• Captures semantic meaning
• Improves NLP model performance
• Handles large text efficiently

7. Limitations

• Requires large datasets
• Computationally expensive
• May capture bias in data

Conclusion:

Coreference resolution techniques were implemented to identify relationships between entities across sentences, improving text coherence and understanding. Word embedding techniques transformed words into meaningful numerical representations, capturing semantic relationships and enhancing the performance of NLP models.

Date:	MODULE 6: NLP CASE STUDIES & TRANSFORMERS

Objective:

• To explore language-oriented case studies and challenges
• To apply machine learning techniques using pretrained transformer models

Problem Statement:

Given real-world NLP problems, perform:

1. Analysis of language challenges through case studies
2. Apply machine learning techniques using pretrained transformer models

Tools Required:

• Python 3.x
• Libraries:
o transformers
o torch / tensorflow
o nltk

Description:

1. Language-Oriented Case Studies

Case studies involve analyzing real-world problems in NLP to understand challenges and solutions.

Common Challenges

• Ambiguity in language
• Sarcasm detection
• Multilingual processing
• Handling noisy data
• Context understanding

2. Examples of Case Studies

• Sentiment analysis of product reviews
• Spam detection in emails
• Chatbot response generation
• Fake news detection

3. Machine Learning in NLP

Machine learning techniques enable systems to learn patterns from data and make predictions.

4. Pretrained Transformers

Transformers are advanced deep learning models used for NLP tasks. They are pretrained on large datasets and fine-tuned for specific tasks.

Popular Transformer Models

• BERT (Bidirectional Encoder Representations from Transformers)
• GPT (Generative Pretrained Transformer)
• RoBERTa

5. Applications of Transformers

• Text classification
• Question answering
• Machine translation
• Text summarization

6. Advantages

• High accuracy
• Handles context effectively
• Pretrained models reduce training time

7. Limitations

• Requires high computational resources
• Large model size
• May produce biased outputs

Conclusion:

Language-oriented case studies highlighted various real-world challenges in natural language processing, such as ambiguity and context understanding. Pretrained transformer models were applied to address these challenges effectively using machine learning techniques, resulting in improved accuracy and performance in NLP tasks.